완전한 트랜스포머 모델 통합 구현

개요

이제 지금까지 학습한 모든 구성 요소들을 통합하여 완전한 트랜스포머 모델을 구현해보겠습니다. 이 문서에서는 어텐션 메커니즘, 포지셔널 인코딩, 인코더-디코더 아키텍처, 그리고 모든 보조 구성 요소들을 하나의 완전한 모델로 결합합니다.

전체 모델 아키텍처

트랜스포머 모델은 다음과 같은 주요 구성 요소들로 이루어집니다:

1. **입력 임베딩 레이어**: 토큰을 벡터로 변환
2. **포지셔널 인코딩**: 순서 정보 추가
3. **인코더 스택**: 6개의 인코더 레이어
4. **디코더 스택**: 6개의 디코더 레이어
5. **출력 레이어**: 최종 예측을 위한 선형 변환

필요한 라이브러리 임포트

```
import torch

import torch.nn as nn

import torch.nn.functional as F

import math

import copy
```

```python
from typing import Optional, Tuple
```

1. 기본 구성 요소들 재정의

먼저 이전 문서들에서 구현한 핵심 구성 요소들을 통합합니다:

1.1 스케일드 닷-프로덕트 어텐션

```python
def  scaled_dot_product_attention(query, key, value, mask=None, dropout=None):
"""
스케일드 닷-프로덕트 어텐션 구현
Args:
query: 쿼리 텐서 [batch_size, seq_len, d_model]
key: 키 텐서 [batch_size, seq_len, d_model]
value: 밸류 텐서 [batch_size, seq_len, d_model]
mask: 마스킹 텐서 (선택적)
dropout: 드롭아웃 레이어 (선택적)
Returns:
어텐션 출력과 어텐션 가중치
"""
d_k = query.size(-1)
# 어텐션 스코어 계산: Q * K^T / sqrt(d_k)
scores = torch.matmul(query, key.transpose(-2, -1)) / math.sqrt(d_k)
# 마스킹 적용 (필요한 경우)
```

```
if mask is not None:
scores = scores.masked_fill(mask == 0, -1e9)
# 소프트맥스로 확률 분포 변환
attention_weights = F.softmax(scores, dim=-1)
# 드롭아웃 적용 (훈련 시)
if dropout is not None:
attention_weights = dropout(attention_weights)
# 가중합 계산
output = torch.matmul(attention_weights, value)
return output, attention_weights
```

1.2 멀티헤드 어텐션

```
class MultiHeadAttention(nn.Module):
"""
멀티헤드 어텐션 구현
여러 개의 어텐션 헤드를 병렬로 실행하여 다양한 관점에서 정보를 처리
"""
def  __init__(self, d_model, num_heads, dropout=0.1):
super(MultiHeadAttention, self).__init__()
assert d_model % num_heads == 0
self.d_model = d_model
self.num_heads = num_heads
```

```
self.d_k = d_model // num_heads

# Q, K, V를 위한 선형 변환 레이어들

self.w_q = nn.Linear(d_model, d_model)

self.w_k = nn.Linear(d_model, d_model)

self.w_v = nn.Linear(d_model, d_model)

self.w_o = nn.Linear(d_model, d_model)

self.dropout = nn.Dropout(dropout)

def  forward(self, query, key, value, mask=None):

batch_size = query.size(0)

# 1. Q, K, V 선형 변환 및 헤드별 분할

Q = self.w_q(query).view(batch_size, -1, self.num_heads, self.d_k).transpose(1,

K = self.w_k(key).view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)

V = self.w_v(value).view(batch_size, -1, self.num_heads, self.d_k).transpose(1,

# 2. 각 헤드에서 어텐션 계산

attention_output, attention_weights = scaled_dot_product_attention(

Q, K, V, mask, self.dropout

)

# 3. 헤드들을 다시 결합

attention_output = attention_output.transpose(1, 2).contiguous().view(

batch_size, -1, self.d_model

)

# 4. 최종 선형 변환

output = self.w_o(attention_output)
```

```
return output, attention_weights
```

1.3 포지션별 피드포워드 네트워크

```
class PositionwiseFeedForward(nn.Module):
"""
포지션별 피드포워드 네트워크
각 위치에서 독립적으로 적용되는 2층 완전연결 네트워크
"""
def  __init__(self, d_model, d_ff, dropout=0.1):
super(PositionwiseFeedForward, self).__init__()
self.w_1 = nn.Linear(d_model, d_ff)
self.w_2 = nn.Linear(d_ff, d_model)
self.dropout = nn.Dropout(dropout)
def  forward(self, x):
# FFN(x) = max(0, xW1 + b1)W2 + b2
return  self.w_2(self.dropout(F.relu(self.w_1(x))))
```

1.4 포지셔널 인코딩

```
class PositionalEncoding(nn.Module):
"""
사인/코사인 함수를 사용한 포지셔널 인코딩
```

```
각 위치에 고유한 패턴을 부여하여 순서 정보를 제공
"""
def __init__(self, d_model, max_len=5000):
super(PositionalEncoding, self).__init__()
# 포지셔널 인코딩 테이블 생성
pe = torch.zeros(max_len, d_model)
position = torch.arange(0, max_len).unsqueeze(1).float()
# 각 차원별로 서로 다른 주파수 사용
div_term = torch.exp(torch.arange(0, d_model, 2).float() *
-(math.log(10000.0) / d_model))
# 짝수 인덱스에는 sin, 홀수 인덱스에는 cos 적용
pe[:, 0::2] = torch.sin(position * div_term)
pe[:, 1::2] = torch.cos(position * div_term)
# 배치 차원 추가 및 버퍼로 등록
pe = pe.unsqueeze(0)
self.register_buffer('pe', pe)
def forward(self, x):
# 입력에 포지셔널 인코딩 추가
return x + self.pe[:, :x.size(1)]
```

2. 인코더 구현

2.1 인코더 레이어

```
class EncoderLayer(nn.Module):
"""
트랜스포머 인코더의 단일 레이어
멀티헤드 어텐션 + 피드포워드 네트워크 + 잔차 연결 + 레이어 정규화
"""
def __init__(self, d_model, num_heads, d_ff, dropout=0.1):
super(EncoderLayer, self).__init__()
self.self_attention = MultiHeadAttention(d_model, num_heads, dropout)
self.feed_forward = PositionwiseFeedForward(d_model, d_ff, dropout)
self.norm1 = nn.LayerNorm(d_model)
self.norm2 = nn.LayerNorm(d_model)
self.dropout = nn.Dropout(dropout)
def forward(self, x, mask=None):
# 1. 셀프 어텐션 + 잔차 연결 + 레이어 정규화
attn_output, _ = self.self_attention(x, x, x, mask)
x = self.norm1(x + self.dropout(attn_output))
# 2. 피드포워드 + 잔차 연결 + 레이어 정규화
ff_output = self.feed_forward(x)
x = self.norm2(x + self.dropout(ff_output))
return x
```

2.2 인코더 스택

```python
class Encoder(nn.Module):
"""
N개의 인코더 레이어로 구성된 인코더 스택
"""
def __init__(self, layer, num_layers):
super(Encoder, self).__init__()
# 동일한 레이어를 N번 복사 (깊은 복사 사용)
self.layers = nn.ModuleList([copy.deepcopy(layer) for _ in range(num_layers)])
self.norm = nn.LayerNorm(layer.self_attention.d_model)
def forward(self, x, mask=None):
# 각 레이어를 순차적으로 통과
for layer in self.layers:
x = layer(x, mask)
# 최종 레이어 정규화
return self.norm(x)
```

3. 디코더 구현

3.1 디코더 레이어

```python
class DecoderLayer(nn.Module):
"""
트랜스포머 디코더의 단일 레이어
```

```
마스크드 셀프 어텐션 + 인코더-디코더 어텐션 + 피드포워드 네트워크

"""

def  __init__(self, d_model, num_heads, d_ff, dropout=0.1):

super(DecoderLayer, self).__init__()

self.self_attention = MultiHeadAttention(d_model, num_heads, dropout)

self.encoder_attention = MultiHeadAttention(d_model, num_heads, dropout)

self.feed_forward = PositionwiseFeedForward(d_model, d_ff, dropout)

self.norm1 = nn.LayerNorm(d_model)

self.norm2 = nn.LayerNorm(d_model)

self.norm3 = nn.LayerNorm(d_model)

self.dropout = nn.Dropout(dropout)

def  forward(self, x, encoder_output, src_mask=None, tgt_mask=None):

# 1. 마스크드 셀프 어텐션 (미래 토큰 참조 방지)

self_attn_output, _ = self.self_attention(x, x, x, tgt_mask)

x = self.norm1(x + self.dropout(self_attn_output))

# 2. 인코더-디코더 어텐션 (인코더 출력 참조)

enc_attn_output, _ = self.encoder_attention(x, encoder_output, encoder_output, s

x = self.norm2(x + self.dropout(enc_attn_output))

# 3. 피드포워드 네트워크

ff_output = self.feed_forward(x)

x = self.norm3(x + self.dropout(ff_output))

return x
```

3.2 디코더 스택

```
class Decoder(nn.Module):
"""
N개의 디코더 레이어로 구성된 디코더 스택
"""
def __init__(self, layer, num_layers):
super(Decoder, self).__init__()
self.layers = nn.ModuleList([copy.deepcopy(layer) for _ in range(num_layers)])
self.norm = nn.LayerNorm(layer.self_attention.d_model)
def forward(self, x, encoder_output, src_mask=None, tgt_mask=None):
# 각 레이어를 순차적으로 통과
for layer in self.layers:
x = layer(x, encoder_output, src_mask, tgt_mask)
return self.norm(x)
```

4. 임베딩 레이어

4.1 토큰 임베딩

```
class TokenEmbedding(nn.Module):
"""
토큰을 고정 크기 벡터로 변환하는 임베딩 레이어
```

```
"""
def __init__(self, vocab_size, d_model):
super(TokenEmbedding, self).__init__()
self.embedding = nn.Embedding(vocab_size, d_model)
self.d_model = d_model
def forward(self, x):
# 임베딩 벡터에 sqrt(d_model)을 곱해 스케일 조정
return self.embedding(x) * math.sqrt(self.d_model)
```

5. 완전한 트랜스포머 모델

이제 모든 구성 요소를 통합하여 완전한 트랜스포머 모델을 구현합니다:

```
class Transformer(nn.Module):
"""
완전한 트랜스포머 모델
인코더-디코더 아키텍처를 사용한 시퀀스-투-시퀀스 모델
"""
def __init__(self, src_vocab_size, tgt_vocab_size, d_model=512, num_heads=8,
num_layers=6, d_ff=2048, max_len=5000, dropout=0.1):
super(Transformer, self).__init__()
self.d_model = d_model
# 임베딩 레이어들
self.src_embedding = TokenEmbedding(src_vocab_size, d_model)
```

```
self.tgt_embedding = TokenEmbedding(tgt_vocab_size, d_model)

self.positional_encoding = PositionalEncoding(d_model, max_len)

# 인코더와 디코더

encoder_layer = EncoderLayer(d_model, num_heads, d_ff, dropout)

decoder_layer = DecoderLayer(d_model, num_heads, d_ff, dropout)

self.encoder = Encoder(encoder_layer, num_layers)

self.decoder = Decoder(decoder_layer, num_layers)

# 출력 레이어

self.output_projection = nn.Linear(d_model, tgt_vocab_size)

# 드롭아웃

self.dropout = nn.Dropout(dropout)

# 파라미터 초기화

self._init_parameters()

def  _init_parameters(self):

"""

모델 파라미터 초기화

Xavier 초기화를 사용하여 안정적인 학습 보장

"""

for p in  self.parameters():

if p.dim() > 1:

nn.init.xavier_uniform_(p)

def  make_src_mask(self, src):

"""
```

```
소스 시퀀스용 패딩 마스크 생성

패딩 토큰(0)을 마스킹하여 어텐션에서 제외

"""

src_mask = (src != 0).unsqueeze(1).unsqueeze(2)

return src_mask

def  make_tgt_mask(self, tgt):

"""

타겟 시퀀스용 마스크 생성

패딩 마스크 + 미래 토큰 마스크 결합

"""

tgt_len = tgt.size(1)

# 패딩 마스크

tgt_pad_mask = (tgt != 0).unsqueeze(1).unsqueeze(3)

# 미래 토큰 마스크 (하삼각 행렬)

tgt_sub_mask = torch.tril(torch.ones(tgt_len, tgt_len)).bool()

# 두 마스크 결합

tgt_mask = tgt_pad_mask & tgt_sub_mask

return tgt_mask

def  forward(self, src, tgt):

"""

트랜스포머 모델의 순전파

Args:

src: 소스 시퀀스 [batch_size, src_len]
```

```
tgt: 타겟 시퀀스 [batch_size, tgt_len]

Returns:

출력 로짓 [batch_size, tgt_len, tgt_vocab_size]

"""

# 마스크 생성

src_mask = self.make_src_mask(src)

tgt_mask = self.make_tgt_mask(tgt)

# 소스 임베딩 + 포지셔널 인코딩

src_embedded = self.dropout(self.positional_encoding(self.src_embedding(src)))

# 타겟 임베딩 + 포지셔널 인코딩

tgt_embedded = self.dropout(self.positional_encoding(self.tgt_embedding(tgt)))

# 인코더 통과

encoder_output = self.encoder(src_embedded, src_mask)

# 디코더 통과

decoder_output = self.decoder(tgt_embedded, encoder_output, src_mask, tgt_mask)

# 최종 출력 레이어

output = self.output_projection(decoder_output)

return output

def  encode(self, src):

"""

인코더만 실행 (추론 시 사용)

"""

src_mask = self.make_src_mask(src)
```

```
src_embedded = self.dropout(self.positional_encoding(self.src_embedding(src)))
return  self.encoder(src_embedded, src_mask)
def  decode(self, tgt, encoder_output, src_mask):
"""
디코더만 실행 (추론 시 사용)
"""
tgt_mask = self.make_tgt_mask(tgt)
tgt_embedded = self.dropout(self.positional_encoding(self.tgt_embedding(tgt)))
return  self.decoder(tgt_embedded, encoder_output, src_mask, tgt_mask)
```

6. 모델 사용 예제

6.1 모델 인스턴스 생성

```
# 모델 하이퍼파라미터 설정
src_vocab_size = 10000  # 소스 언어 어휘 크기
tgt_vocab_size = 8000  # 타겟 언어 어휘 크기
d_model = 512  # 모델 차원
num_heads = 8  # 어텐션 헤드 수
num_layers = 6  # 인코더/디코더 레이어 수
d_ff = 2048  # 피드포워드 네트워크 차원
max_len = 5000  # 최대 시퀀스 길이
dropout = 0.1  # 드롭아웃 비율
```

```
# 모델 생성
model = Transformer(
src_vocab_size=src_vocab_size,
tgt_vocab_size=tgt_vocab_size,
d_model=d_model,
num_heads=num_heads,
num_layers=num_layers,
d_ff=d_ff,
max_len=max_len,
dropout=dropout
)

print(f"모델 파라미터 수: {sum(p.numel() for p in model.parameters()):,}")
```

6.2 순전파 예제

```
# 예제 입력 데이터
batch_size = 32
src_len = 20
tgt_len = 25

# 랜덤 입력 생성 (실제로는 토크나이저 출력 사용)
```

```
src = torch.randint(1, src_vocab_size, (batch_size, src_len))

tgt = torch.randint(1, tgt_vocab_size, (batch_size, tgt_len))


# 모델 순전파

with torch.no_grad():

output = model(src, tgt)

print(f"출력 형태: {output.shape}") # [32, 25, 8000]

# 예측 확률 계산

predictions = F.softmax(output, dim=-1)

print(f"예측 확률 형태: {predictions.shape}")
```

6.3 추론 과정 예제

```
def  greedy_decode(model, src, max_len, start_token, end_token):

"""

그리디 디코딩을 사용한 추론

각 단계에서 가장 확률이 높은 토큰 선택

"""

model.eval()

# 인코더 실행

encoder_output = model.encode(src)

src_mask = model.make_src_mask(src)

# 디코더 입력 초기화
```

```
tgt = torch.zeros(1, 1).fill_(start_token).type_as(src)

for i in  range(max_len - 1):

# 현재까지의 출력으로 다음 토큰 예측

decoder_output = model.decode(tgt, encoder_output, src_mask)

next_token_logits = model.output_projection(decoder_output[:, -1])

next_token = next_token_logits.argmax(dim=-1)

# 다음 토큰을 시퀀스에 추가

tgt = torch.cat([tgt, next_token.unsqueeze(1)], dim=1)

# 종료 토큰이 나오면 중단

if next_token.item() == end_token:

break

return tgt



# 추론 예제

src_example = torch.randint(1, src_vocab_size, (1, 15))

decoded_sequence = greedy_decode(model, src_example, max_len=30,

start_token=1, end_token=2)

print(f"디코딩된 시퀀스: {decoded_sequence}")
```

7. 모델 분석 및 이해

7.1 파라미터 공유 전략

트랜스포머에서는 다음과 같은 파라미터 공유 전략을 사용할 수 있습니다:

```
def tie_weights(model):
"""
임베딩 레이어와 출력 레이어 간 가중치 공유
메모리 효율성 향상 및 성능 개선 효과
"""
if model.src_embedding.embedding.weight.shape == model.output_projection.weight.
model.output_projection.weight = model.tgt_embedding.embedding.weight
print("타겟 임베딩과 출력 레이어 가중치 공유 완료")


# 가중치 공유 적용
tie_weights(model)
```

7.2 모델 복잡도 분석

```
def analyze_model_complexity(model):
"""
모델의 계산 복잡도와 메모리 사용량 분석
"""
total_params = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"전체 파라미터 수: {total_params:,}")
```

```
print(f"학습 가능한 파라미터 수: {trainable_params:,}")

# 각 구성 요소별 파라미터 수

encoder_params = sum(p.numel() for p in model.encoder.parameters())

decoder_params = sum(p.numel() for p in model.decoder.parameters())

embedding_params = sum(p.numel() for p in model.src_embedding.parameters()) + \

sum(p.numel() for p in model.tgt_embedding.parameters())

print(f"인코더 파라미터: {encoder_params:,}")

print(f"디코더 파라미터: {decoder_params:,}")

print(f"임베딩 파라미터: {embedding_params:,}")


analyze_model_complexity(model)
```

8. 핵심 설계 원칙

8.1 잔차 연결의 중요성

```
# 잔차 연결이 없는 경우와 있는 경우 비교

def  forward_without_residual(self, x):

"""잔차 연결 없는 버전 (권장하지 않음)"""

attn_output, _ = self.self_attention(x, x, x)

x = self.norm1(attn_output) # 잔차 연결 제거

ff_output = self.feed_forward(x)

x = self.norm2(ff_output) # 잔차 연결 제거
```

```
return x


def  forward_with_residual(self, x):
"""잔차 연결 있는 버전 (권장)"""
attn_output, _ = self.self_attention(x, x, x)
x = self.norm1(x + attn_output) # 잔차 연결 유지
ff_output = self.feed_forward(x)
x = self.norm2(x + ff_output) # 잔차 연결 유지
return x
```

8.2 레이어 정규화 위치

```
# Pre-LN vs Post-LN 구조 비교
class PreLNEncoderLayer(nn.Module):
"""Pre-Layer Normalization (더 안정적인 학습)"""
def  forward(self, x, mask=None):
# 정규화를 먼저 적용
attn_output, _ = self.self_attention(self.norm1(x), self.norm1(x), self.norm1(x)
x = x + attn_output
ff_output = self.feed_forward(self.norm2(x))
x = x + ff_output
return x
```

```
class PostLNEncoderLayer(nn.Module):
"""Post-Layer Normalization (원본 논문 구조)"""
def  forward(self, x, mask=None):
# 잔차 연결 후 정규화 적용
attn_output, _ = self.self_attention(x, x, x, mask)
x = self.norm1(x + attn_output)
ff_output = self.feed_forward(x)
x = self.norm2(x + ff_output)
return x
```

9. 성능 최적화 팁

9.1 메모리 효율적인 어텐션

```
def  memory_efficient_attention(query, key, value, mask=None, chunk_size=1024):
"""
메모리 효율적인 어텐션 계산
긴 시퀀스에서 메모리 사용량 감소
"""
batch_size, num_heads, seq_len, d_k = query.shape
if seq_len <= chunk_size:
return scaled_dot_product_attention(query, key, value, mask)
# 청크 단위로 분할하여 계산
```

```
output = torch.zeros_like(query)

for i in  range(0, seq_len, chunk_size):

end_i = min(i + chunk_size, seq_len)

chunk_output, _ = scaled_dot_product_attention(

query[:, :, i:end_i], key, value,

mask[:, :, i:end_i] if mask is  not None else None

)

output[:, :, i:end_i] = chunk_output

return output, None
```

9.2 그래디언트 체크포인팅

```
from torch.utils.checkpoint import checkpoint


class CheckpointedEncoderLayer(nn.Module):

"""

그래디언트 체크포인팅을 사용한 메모리 효율적인 인코더 레이어

"""

def  forward(self, x, mask=None):

# 메모리 절약을 위해 체크포인팅 사용

return checkpoint(self._forward, x, mask)

def  _forward(self, x, mask):

attn_output, _ = self.self_attention(x, x, x, mask)
```

```
x = self.norm1(x + self.dropout(attn_output))

ff_output = self.feed_forward(x)

x = self.norm2(x + self.dropout(ff_output))

return x
```

10. 요약

이 문서에서는 트랜스포머의 모든 구성 요소를 통합하여 완전한 모델을 구현했습니다. 주요 특징은 다음과 같습니다:

핵심 구성 요소

- **멀티헤드 어텐션**: 다양한 관점에서 정보 처리
- **포지셔널 인코딩**: 순서 정보 제공
- **인코더-디코더 구조**: 입력 이해와 출력 생성 분리
- **잔차 연결**: 그래디언트 흐름 개선
- **레이어 정규화**: 학습 안정성 향상

설계 원칙

- **모듈화**: 각 구성 요소의 독립적 구현
- **확장성**: 다양한 크기와 구성으로 조정 가능
- **효율성**: 메모리와 계산 최적화 고려
- **안정성**: 수치적 안정성과 학습 안정성 보장

이제 완전한 트랜스포머 모델을 이해하고 구현할 수 있게 되었습니다. 다음 단계에서는 이 모델을 실제로 훈련하고 최적화하는 방법을 학습하게 됩니다.

다음 문서: 10_optimization_training.md - 최적화와 훈련 전략

이전 문서: 08_layer_normalization.md - 레이어 정규화와 잔차 연결

네비게이션

- **이전**: 레이어 정규화와 잔차 연결
- **다음**: 최적화와 훈련 전략
- **목차**: README.md
- **전체 목록**: 네비게이션